

Confidential

레이다 SW 학습 과제 3

표적 궤적 모의

Target Trajectory Simulation

Janghwan Kim (Harley)

발표 대본

표지 · 약 20초

안녕하십니까. 과제 3, 표적 궤적 모의 발표를 시작하겠습니다.

내용은 세 덩어리입니다. 표적을 어떤 구조체로 정의했는지, 그 표적을 시간에 따라 어떻게 움직였는지, 그리고 명세 조건으로 돌린 결과가 맞는지입니다.

INDEX

표적 궤적 모의
TargetSim

Janghwan Kim (Harley)

01

표적 구조체 설계

표적 하나를 어떤 숫자로 적을 것인가
기동을 어떻게 표현할 것인가

02

표적 궤적 모의 구현

동체 속도를 ECEF 속도로 바꾸는 방법
한 스텝을 전진시키는 방법

03

시뮬레이션 결과

명세 조건 60 초 결과와 계산
정확도 확인과 기동 시연

2

발표 대본

목차 · 약 40초

먼저 구조체 설계입니다. 표적 하나를 숫자 몇 개로 적어야 충분한지, 그리고 "기동"이라는 말을 코드가 알아들을 수 있는 값으로 어떻게 바꿨는지를 보겠습니다.

둘째가 궤적 모의입니다. 여기에 과제에서 추가로 고민해 보라고 한 항목, 동체 속도를 ECEF 속도 성분으로 바꾸는 방법이 들어 있습니다.

셋째가 결과입니다. 명세 조건 그대로 60초를 돌린 결과를 보여 드리고, 그 숫자가 맞는지 손으로 계산한 값과 맞춰 보겠습니다.

수학은 삼각함수와 행렬 곱셈 정도만 나오고, 그마저도 필요한 자리에서 다시 설명하겠습니다.

들어가며

과제가 요구한 것

「과제 3 내용」 문서의 요구사항을 네 덩어리로 묶고, 발표도 그 순서로 갑니다.

표적 구조체 정의

표적 최대 10 개, 표적마다 기동 최대 30 개.
위치는 위도 · 경도 · 고도, 속도는 동체 속도 ($V_{heading}$) 한 개로 입력받을 것.

표적 상태 갱신

위치와 속도는 ECEF 좌표계에서 갱신할 것.
추가 고민 — $V_{heading}$ 을 ECEF 속도 V_x, V_y, V_z 로 바꾸는 방법.

시뮬레이션 조건

60 초 동안 0.1 초 간격.
플랫폼 1 + 대함 표적 1 + 대공 표적 1.
제공받은 좌표변환 코드를 그대로 사용.

결과 제시

궤적을 LLA 로 출력하고 그림으로 표현.
구조체 정의 · 구현 방법 · 결과를 코드와 함께 설명.

코드는 Part 2 에서 쓰던 좌표변환 라이브러리를 한 줄도 고치지 않고 그대로 가져다 썼습니다.
새로 만든 것은 TargetSim.c 하나 — 표적을 시간에 따라 움직이는 부분입니다.

3

발표 대본

도입 · 약 55초

과제 문서의 요구사항을 네 덩어리로 묶었습니다.

첫째, 구조체 정의입니다. 표적은 최대 10개, 표적마다 기동은 최대 30개까지 설정할 수 있어야 하고, 위치는 위도 경도 고도로, 속도는 동체 속도 하나로 받으라고 되어 있습니다.

둘째, 상태 갱신은 ECEF 좌표계에서 하라는 것입니다. 여기에 "동체 속도를 ECEF 성분으로 바꾸는 방법을 고민해 보라" 는 항목이 괄호로 붙어 있습니다. 안 해도 된다고 적혀 있지만, 사실 이걸 안 풀면 구현 자체가 안 되는 부분이 라 그대로 다뤘습니다.

셋째가 시뮬레이션 조건, 넷째가 결과 제시입니다.

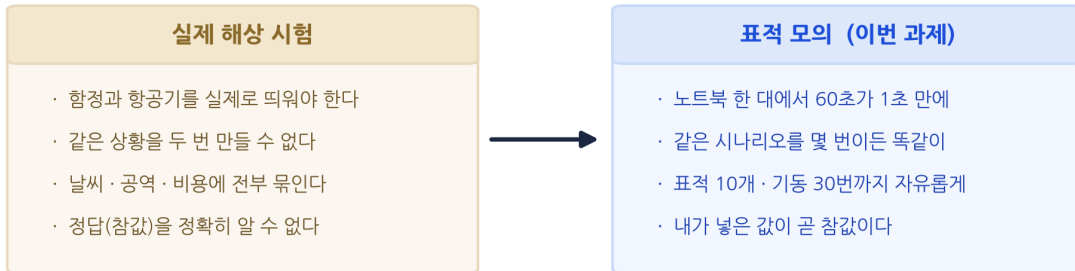
한 가지 미리 말씀드리면, 좌표변환 코드는 제공받은 것을 한 줄도 고치지 않고 그대로 썼습니다. 새로 만든 것은 TargetSim.c 하나입니다.

들어가며

표적을 왜 가짜로 만드나요?

레이다 소프트웨어를 만들려면 먼저 "표적이 이렇게 움직였다" 는 데이터가 있어야 합니다.

같은 데이터를 얻는 두 가지 길



데이터처리 소프트웨어가 받는 입력은 어느 쪽이든 똑같다
— 시각마다 찍힌 표적의 위도 · 경도 · 고도

4

발표 대본

도입 · 약 50초

질문 하나로 시작하겠습니다. 표적을 왜 가짜로 만들까요.

레이다 데이터처리 소프트웨어를 만들려면 입력이 있어야 합니다. 그 입력은 "몇 시 몇 분 몇 초에 표적이 어디 있었다" 는 기록입니다. 이것 얻으려면 배를 띄우고 비행기를 띄워야 합니다.

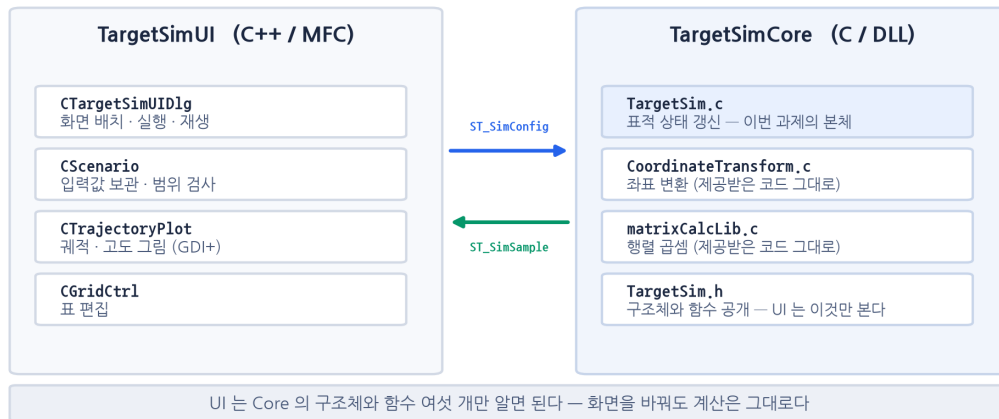
문제는 세 가지입니다. 돈이 많이 들고, 같은 상황을 두 번 만들 수 없고, 무엇보다 정답을 모릅니다. 실제 시험에서는 표적이 정확히 어디 있었는지를 다른 장비로 또 재야 하는데, 그 장비도 오차가 있습니다.

모의는 반대입니다. 제가 "여기서 출발해서 저렇게 꺾어라" 하고 넣은 값이 곧 정답입니다. 그래서 알고리즘이 틀렸는지 데이터가 틀렸는지 헷갈릴 일이 없습니다.

들어가며

만든 것 — 계산은 C, 화면은 MFC

계산 엔진과 화면을 나눠 두면 화면이 바뀌어도 계산은 건드릴 일이 없습니다.



5

발표 대본

도입 · 약 50초

만든 물건은 두 덩어리입니다.

오른쪽 TargetSimCore 가 계산 엔진이고 순수 C 입니다. 이 중에서 CoordinateTransform 과 matrixCalcLib 는 제공받은 코드 그대로고, 제가 새로 쓴 것은 맨 위 TargetSim.c 하나입니다.

왼쪽 TargetSimUI 는 MFC 로 만든 화면입니다. 표에서 값을 고치고, 궤적을 보고, CSV 로 내보내는 일을 합니다.

둘 사이에 오가는 것은 구조체 두 개뿐입니다. 설정을 담은 ST_SimConfig 를 넣으면 시각마다의 상태인 ST_SimSample 이 나옵니다. UI 는 Core 가 안에서 뭘 하는지 전혀 모르고, Core 도 화면이 어떻게 생겼는지 모릅니다.

이렇게 나눈 덕을 실제로 봤습니다. 발표 자료를 만들면서 리눅스에서 gcc 로 Core 만 따로 컴파일해서 돌렸는데, MFC 쪽은 건드릴 필요가 없었습니다.

01 표적 구조체 설계

Target Structure Design

- 표적 하나를 숫자 몇 개로 적어야 충분한가
- "기동"이라는 말을 코드가 알아들을 수 있는 값으로 바꾸기
- 설정 (입력) 과 상태 (출력) 를 나눈 이유

6

발표 대본

장절 · 약 25초

첫 번째 장입니다. 표적 하나를 어떤 숫자들로 적을지 정하는 일입니다.

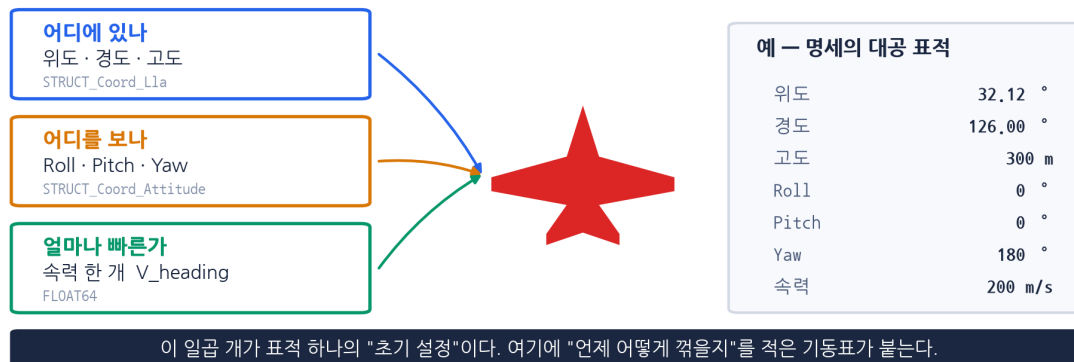
이 부분은 코드를 짜기 전에 종이에서 결정해야 하는 것이고, 한 번 정하면 나중에 바꾸기가 제일 어려운 부분이기도 합니다.

01. 표적 구조체 설계

표적 하나를 숫자로 적으면

지금 어디에 있고, 어디를 보고 있고, 얼마나 빠르냐 — 이 세 가지면 다음 순간을 계산할 수 있습니다.

표적 하나는 숫자 일곱 개로 적힌다



7

발표 대본

구조체 · 약 55초

표적 하나를 다음 순간으로 옮기려면 지금 무엇을 알아야 할까요.

셋입니다. 지금 어디에 있는지, 어디를 보고 있는지, 얼마나 빠르냐.

어디에 있는지는 위도 경도 고도 세 개입니다. 어디를 보고 있는지는 Roll, Pitch, Yaw 세 개, 얼마나 빠르냐는 속력 한 개입니다. 합쳐서 일곱 개입니다.

오른쪽이 명세에 나온 대공 표적을 그대로 적은 것입니다. 위도 32.12도, 경도 126도, 고도 300미터, 자세각은 전부 0인데 Yaw 만 180도, 속력은 초속 200미터.

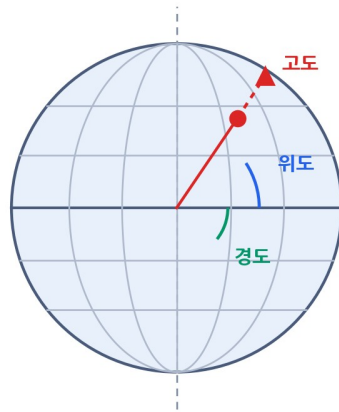
Yaw 180도는 남쪽을 본다는 뜻입니다. 북쪽이 0도이고 시계 방향으로 잹니다.

이 일곱 개가 표적 하나의 초기 설정이고, 여기에 언제 어떻게 꺾을지를 적은 기동표가 따로 붙습니다.

01. 표적 구조체 설계

위치는 위도 · 경도 · 고도로 받는다

명세가 정한 입력 형식이기도 하고, 사람이 지도에서 바로 찍어 볼 수 있는 값이기도 합니다.

**위도 (Latitude)**

적도에서 남북으로 잰 각
-90° ~ +90°

경도 (Longitude)

영국 그리니치에서 동서로 잰 각
-180° ~ +180°

고도 (Altitude)

지표면에서 위로 잰 높이
미터

사람이 읽기 좋은 좌표 — 그래서 입력과 출력은 LLA 로 받고 쓴다

8

발표 대본

구조체 · 약 45초

위치를 위도 경도 고도로 받는 것은 우선 명세가 그렇게 정해서입니다. 그런데 안 그렇게 정했어도 이렇게 했을 것 같습니다.

위도는 적도에서 남북으로 잰 각이고, 경도는 영국 그리니치에서 동서로 잰 각, 고도는 지표면에서 위로 잰 높이입니다.

이 값이 좋은 이유는 사람이 바로 확인할 수 있어서입니다. "위도 32.12, 경도 126.0" 을 지도에 찍으면 제주도 서쪽 바다가 나옵니다. 여기가 맞는 자리인지 눈으로 볼 수 있습니다.

같은 점을 ECEF 로 적으면 x 는 3백 18만, y 는 4백 37만, z 는 3백 37만 미터쯤 되는데, 이 숫자만 보고는 맞는지 틀렸는지 알 수가 없습니다.

그래서 입력과 출력은 LLA 로 받고 쓰고, 계산만 다른 좌표계에서 합니다.

01. 표적 구조체 설계

속도는 크기 하나만 받는다

방향은 자세각이 이미 들고 있으므로, 속도까지 방향을 갖게 하면 두 개가 서로 어긋납니다.

속도를 어떻게 입력받을 것인가

x, y, z 세 숫자로 받으면

"북서쪽으로 초속 30 m" 를
사람이 직접 세 숫자로 쪼개야 한다
세 숫자를 조금 잘못 넣으면
속력이 30 이 아니라 31.4 가 된다
자세각과 따로 놀아서, 기수는 서쪽인데
속도는 동쪽인 표적도 만들어진다

속력 하나 + 자세각으로 받으면

"기수 방향으로 초속 30 m" 그대로 적는다
조종사가 계기판에서 읽는 값과 같다
회전만 시키므로 속력은 절대 안 변한다
(회전은 길이를 바꾸지 않는다)
기동은 자세각만 돌리면 되고
속도 방향은 저절로 따라온다

명세도 "동체 속도(Heading 방향, V_heading) 성분으로 입력받도록" 을 요구한다

9

발표 대본

구조체 · 약 55초

속도는 조금 더 생각할 거리가 있었습니다.

속도는 원래 방향이 있는 값이니까 x, y, z 세 개로 받는 게 자연스러워 보입니다. 그런데 그렇게 하면 문제가 생깁니다.

첫째, 사람이 "북서쪽으로 초속 30미터" 를 직접 세 숫자로 쪼개야 합니다. 둘째, 그 세 숫자를 조금만 잘못 넣으면 속력이 30이 아니라 31.4가 됩니다. 셋째가 제일 곤란한데, 자세각과 따로 놀아서 기수는 서쪽인데 속도는 동쪽인 표적이 만들어집니다.

명세가 정한 방식은 반대입니다. 속력 하나만 받고, 방향은 자세각이 담당합니다. 조종사가 계기판에서 읽는 값이 딱 이 형태입니다.

이렇게 하면 속력이 절대 안 변합니다. 자세각을 돌리는 건 회전인데, 회전은 길이를 안 바꾸기 때문입니다. 실제로 확인한 숫자는 뒤에서 보여 드리겠습니다.

01. 표적 구조체 설계

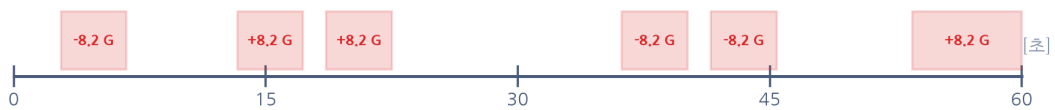
기동을 어떻게 적을 것인가

"12 초부터 16 초까지 오른쪽으로 8G 로 꺾어라" — 이 한 문장을 네 개의 값으로 쪼갭니다.

기동 한 줄에 들어가는 것 네 가지

TurnType	Gravity Value	StartTime	EndTime
어느 축으로 0 없음 / 1 Roll 2 Yaw / 3 Pitch	얼마나 세게 중력가속도의 몇 배 부호가 회전 방향	언제부터 [초]	언제까지 [초]

표적 하나에 이런 줄을 최대 30개까지 — 걸린 구간에서만 기수가 돌아가고, 나머지 시간은 직진이다



↑ 기동 시연 시나리오에서 대공 표적에 걸여 둔 여섯 줄

10

발표 대본

구조체 · 약 55초

다음이 기동입니다. "12초부터 16초까지 오른쪽으로 8G 로 꺾어라" 라는 말을 코드가 알아들을 수 있는 값으로 바꿔야 합니다.

네 개면 됩니다. 어느 축으로 도는지, 얼마나 세게 도는지, 언제부터, 언제까지.

축은 명세가 정한 그대로 0이 회전 없음, 1이 Roll, 2가 Yaw, 3이 Pitch 입니다. 세기는 G 값인데 부호가 방향입니다. 플러스면 오른쪽, 마이너스면 왼쪽입니다.

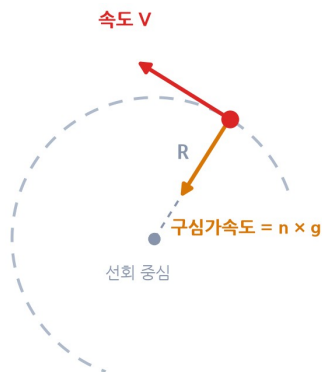
아래 시간축이 실제 시나리오 하나입니다. 60초 중에 여섯 군데만 색이 칠해져 있는데, 저 구간에서만 기수가 돌아가고 나머지는 전부 직진입니다.

한 가지 정해야 했던 것이 구간의 경계입니다. 시작은 포함하고 끝은 포함하지 않는 것으로 했습니다. 그래야 한 기동이 16초에 끝나고 다음 기동이 16초에 시작해도 겹침으로 잡히지 않습니다.

01. 표적 구조체 설계

G 값 하나가 회전 속도를 정한다

왜 각도 (°/s) 대신 G 로 받을까요? 조종사와 기체가 실제로 견디는 한계가 G 이기 때문입니다.



원운동은 식 한 줄이면 끝난다

$$V \times \omega = n \times g$$

속력 × 각속도 = 중력가속도의 n 배

$$\omega = (n \times 9.80665) \div V$$

G 값 하나만 받으면 초당 몇 도를 돌지가 정해진다.

`omega = (gravityValue * G_FORCE) / headingSpeed;`

11

발표 대본

구조체 · 약 60초

왜 각속도를 직접 받지 않고 G 로 받을까요.

이유는 G 가 실제 한계값이기 때문입니다. 전투기는 9G 근처가 한계고 조종사도 그 근처에서 의식을 잃습니다. 대함미사일은 그것보다 훨씬 낮습니다. 시나리오를 만드는 사람은 "이 표적이 낼 수 있는 G 가 얼마인가" 를 기준으로 생각하지, "초당 몇 도" 로 생각하지 않습니다.

식은 한 줄입니다. 원운동에서 구심가속도는 속력 곱하기 각속도입니다. 이게 중력가속도의 n 배라고 하면 V 곱하기 오메가는 n 곱하기 g 가 됩니다.

여기서 오메가만 남기면 n 곱하기 9.80665 나누기 V 입니다. 코드에서도 딱 이 한 줄입니다.

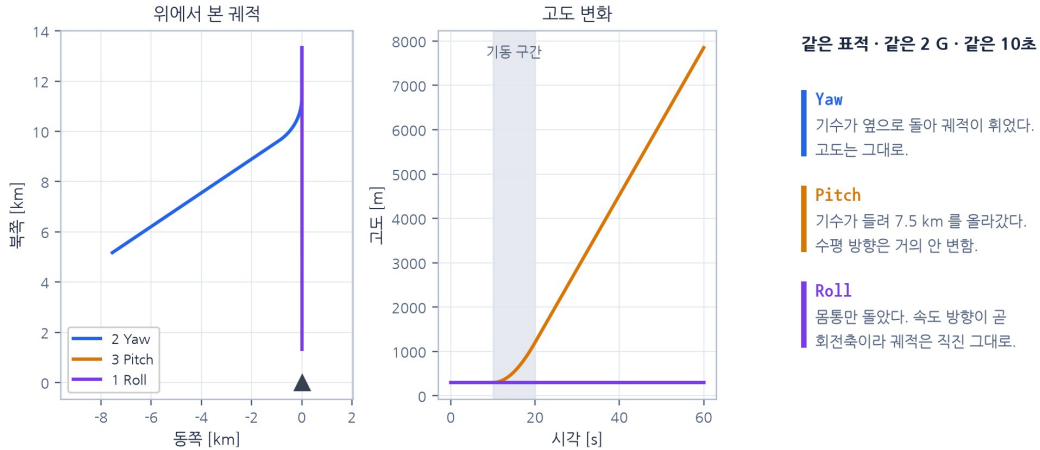
같은 G 라도 빠른 표적이 덜 꺾인다는 점을 여기서 읽을 수 있습니다. 분모에 속력이 있으니까요. 그래서 초속 200미터짜리 대공 표적을 90도 돌리려면 G 를 8 넘게 줘야 합니다.

Roll 은 사실 구심가속도와 아무 상관이 없습니다. 그래도 G 값 하나로 세 축을 다 다루려고 같은 식을 씁니다. 이젠 제가 임의로 정한 규칙이라 주석에 남겨 뒀습니다.

01. 표적 구조체 설계

회전축 세 가지 — 실제로 돌려 보면

같은 표적에 같은 2G를 10초씩, 축만 바꿔 걸어 본 결과입니다.



12

발표 대본

구조체 · 약 60초

축이 셋이라고 말로만 하면 감이 안 오니까, 실제로 돌려 봤습니다.

같은 대공 표적, 같은 2G, 같은 10초입니다. 축만 바꿨습니다.

왼쪽이 위에서 본 궤적입니다. 파란색 Yaw는 기수가 옆으로 돌아서 궤적이 확 휘었습니다. 고도는 오른쪽 그림에서 보시는 것처럼 300미터 그대로입니다.

주황색 Pitch는 반대입니다. 기수가 들어서 10초 동안 7.5킬로미터를 올라갔습니다. 위에서 보면 거의 제자리인데 옆에서 보면 급상승입니다.

보라색 Roll이 재미있습니다. 궤적이 전혀 안 변했습니다. 고도도 그대로입니다.

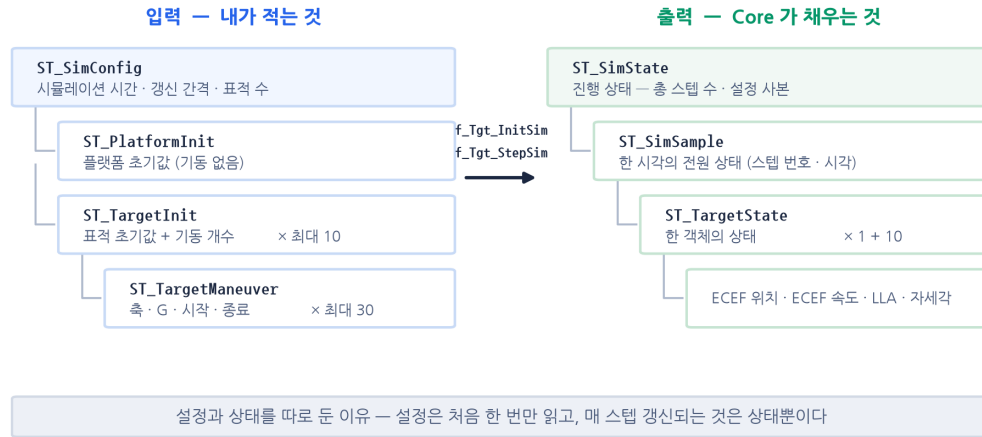
이유는 Roll축이 속도 방향 그 자체이기 때문입니다. 자기가 가는 방향을 축으로 삼아 몸통만 돌리는 거라, 방향이 바뀔 수가 없습니다. 실제 비행기도 Roll만으로는 방향을 못 바꾸고, Roll로 기울인 다음 Pitch를 당겨야 선회가 됩니다.

이 표를 만들면서 Roll 구현이 틀린 줄 알고 한참 봤는데, 물리적으로 맞는 결과였습니다.

01. 표적 구조체 설계

구조체 전체 그림

입력 쪽과 출력 쪽을 완전히 갈라 두었습니다.



13

발표 대본

구조체 · 약 50초

지금까지 이야기한 것을 구조체로 그리면 이렇게 됩니다.

왼쪽이 입력입니다. 맨 위 ST_SimConfig 가 시뮬레이션 시간과 갱신 간격, 표적 수를 들고 있고, 그 안에 플랫폼 하나와 표적 배열이 들어갑니다. 표적마다 기동 배열이 또 들어갑니다. 10개와 30개는 배열 크기로 박아 뒀습니다.

오른쪽이 출력입니다. ST_SimState 가 진행 상태를 들고, 그 안의 ST_SimSample 이 "이 시각의 전역 상태" 한 장입니다. 그 안에 객체마다 ST_TargetState 가 있고, 거기에 ECEF 위치, ECEF 속도, LLA, 자세각이 들어갑니다.

둘을 왜 갈랐냐면, 설정은 처음 한 번만 읽고 매 스텝 바뀌는 것은 상태뿐이기 때문입니다. 섞어 놓으면 "이 값이 내가 넣은 값인지 계산된 값인지" 가 헷갈립니다.

동적 할당은 한 군데도 없습니다. 전부 고정 배열입니다.

01. 표적 구조체 설계

코드로 보는 구조체 ① — 입력

TargetSimCore/include/TargetSim.h

```
// 기동 회전축 . 값은 과제 명세를 그대로 따른다 .
typedef enum
{
    TGT_TURN_NONE = 0,   TGT_TURN_ROLL = 1,
    TGT_TURN_YAW  = 2,   TGT_TURN_PITCH = 3
} EN_TurnType;

// 기동 1건 . gravityValue 의 부호가 회전 방향이다 .
typedef struct
{
    EN_TurnType  enTurnType;
    FLOAT64      gravityValue; // [G]
    FLOAT64      startTime;    // [s]
    FLOAT64      endTime;      // [s]
} ST_TargetManeuver;

typedef struct
{
    STRUCT_Coord_Lla  st_InitLla;
    STRUCT_Coord_Attitude st_InitAtt;
    FLOAT64           headingSpeed; // V_heading
    INT32              nManeuverNum;
    ST_TargetManeuver st_Maneuver[TGT_MAX_MANEUVER_NUM];
} ST_TargetInit;
```

구간은 [시작, 종료)

한 기동의 끝과 다음 기동의 시작이 같아도 겹침이 아니라 이어 붙는 것으로 봅니다.

겹치면 아예 거부

같은 표적에 같은 시각을 덮는 기동이 둘이면 TGT_ERR_MANEUVER_OVERLAP 을 돌려줍니다.

10 × 30 은 배열로 고정

동적 할당을 쓰지 않아 실행 중 메모리 실패가 생길 자리가 없습니다.

14

발표 대본

코드 · 약 65초

실제 헤더 파일입니다.

위가 회전축 enum 입니다. 값을 0, 1, 2, 3 으로 명세와 똑같이 맞췄습니다. 이렇게 해 두면 UI 에서 사용자가 고른 번호를 그대로 넣을 수 있어서 변환표가 필요 없습니다.

가운데가 기동 한 건입니다. 축, G 값, 시작 시각, 종료 시각. 네 개뿐입니다.

아래가 표적 초기 설정입니다. 위치는 STRUCT_Coord_Lla, 자세는 STRUCT_Coord_Attitude 인데 이 둘은 제공 받은 좌표변환 헤더에 이미 있던 구조체입니다. 새로 만들지 않고 그대로 썼습니다. 그 아래 headingSpeed 가 아까 말씀드린 속력 하나입니다.

오른쪽 메모 세 개만 짤겠습니다.

첫째, 구간은 시작 포함, 종료 제외입니다.

둘째, 겹치면 아예 거부합니다. 같은 표적에 같은 시각을 덮는 기동이 둘이면 어느 쪽을 따를지 애매해지니까, 계산을 시작하기 전에 오류 코드를 돌려줍니다.

셋째, 10 곱하기 30 을 배열로 박았습니다. 표적 열 개에 기동 서른 개면 구조체가 10킬로바이트 정도인데, 요즘 기준으로는 작은 값이고 대신 실행 중에 메모리 할당이 실패할 자리가 없습니다.

01. 표적 구조체 설계

코드로 보는 구조체 ② — 출력

설정은 처음 한 번만 읽고, 스텝마다 바뀌는 것은 상태뿐입니다.

```
// 한 시각의 상태. 위치와 속도는 ECEF 에서 갱신하고,
// LLA 는 출력과 다음 스텝의 NED 기준으로 함께 보관한다.
typedef struct
{
    FLOAT64      simTime;      // [s]
    STRUCT_Coord_Rect st_PosEcef; // [m]
    STRUCT_Coord_Rect st_VelEcef; // [m/s]
    STRUCT_Coord_Lla st_Lla;
    STRUCT_Coord_Attitude st_Att;
} ST_TargetState;

// 한 시각의 플랫폼 · 표적 상태.
typedef struct
{
    FLOAT64      simTime;
    INT32        nStepIndex;
    INT32        nTargetNum;
    ST_TargetState st_Platform;
    ST_TargetState st_Target[TGT_MAX_TARGET_NUM];
} ST_SimSample;
```

왜 ECEF 와 LLA 를 둘 다 들고 있나

계산은 ECEF 에서 하는 쪽이 정확하고,
출력과 화면은 LLA 라야 읽힙니다.
스텝마다 한 번만 바뀌 두면 쓸 때마다
다시 변환하지 않아도 됩니다.

플랫폼도 같은 구조체

플랫폼은 기동이 없는 표적일 뿐입니다.
같은 ST_TargetState 를 쓰고 각속도만
0 으로 넣어 같은 경로로 전진시킵니다.

15

발표 대본

코드 · 약 55초

출력 쪽입니다.

위가 객체 하나의 상태입니다. 여기서 눈여겨보실 것은 위치와 속도를 ECEF 로 들고 있으면서 LLA 도 같이 들고 있다는 점입니다.

값이 중복된다고 볼 수도 있는데, 이유가 있습니다. 계산은 ECEF 에서 해야 정확하고, 출력과 화면은 LLA 라야 읽힙니다. 스텝마다 한 번만 바뀌어서 같이 들고 있으면, 쓸 때마다 다시 변환하지 않아도 됩니다.

그리고 사실 LLA 는 다음 스텝 계산에도 필요합니다. 속도를 만들려면 그 지점의 위도와 경도를 알아야 하거든요. 이걸 뒤에서 다시 나옵니다.

아래가 한 시각의 전원 상태입니다. 플랫폼이 따로 한 칸, 표적이 배열로 들어갑니다.

플랫폼도 같은 ST_TargetState 를 씁니다. 플랫폼은 기동이 없는 표적일 뿐이라, 각속도만 0 으로 넣고 똑같은 함수를 통과시킵니다. 코드가 두 벌이 안 됩니다.

02 표적 궤적 모의 구현

Trajectory Propagation

- 기수 방향 속력 하나를 ECEF 속도 세 성분으로 바꾸기
- 0.1 초를 한 번 전진시키는 방법 — 중점법
- 숫자가 깨졌을 때 상태를 지키는 방법

16

발표 대본

장절 · 약 25초

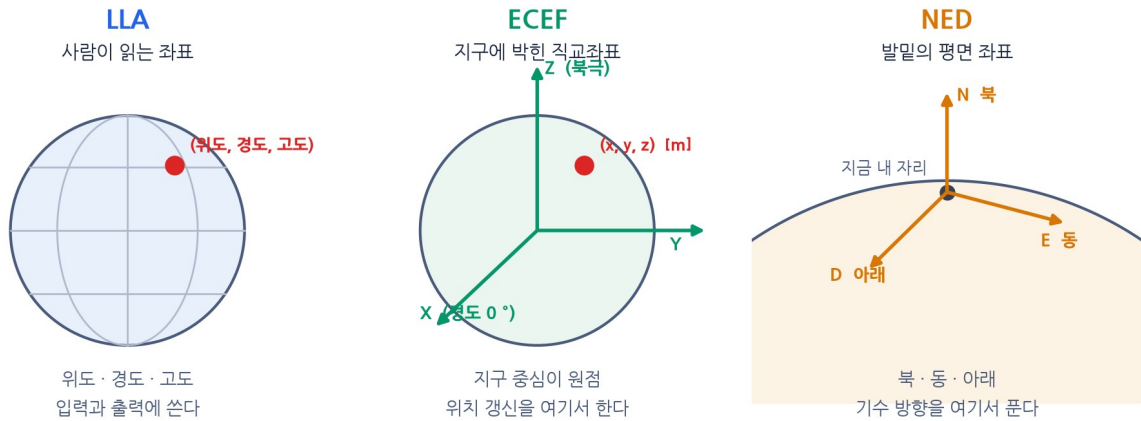
두 번째 장입니다. 구조체에 담긴 값을 실제로 움직이는 부분입니다.

여기가 이번 과제의 핵심이고, 과제에서 추가로 고민해 보라고 한 항목도 이 안에 있습니다.

02. 표적 궤적 모의 구현

쓰는 좌표계는 셋뿐입니다

Part 2 에서 여섯 개를 봤지만, 표적을 움직이는 데 실제로 필요한 것은 이 셋입니다.



17

발표 대본

구현 · 약 50초

Part 2 에서 좌표계를 여섯 개 봤는데, 이번에 실제로 쓰는 것은 셋입니다.

LLA 는 방금 본 위도 경도 고도입니다. 입력과 출력에 씁니다.

ECEF 는 지구 중심을 원점으로 잡은 직교좌표입니다. 축이 지구에 박혀 있어서 지구와 같이 돕니다. 단위는 미터입니다. 위치 갱신을 여기서 합니다.

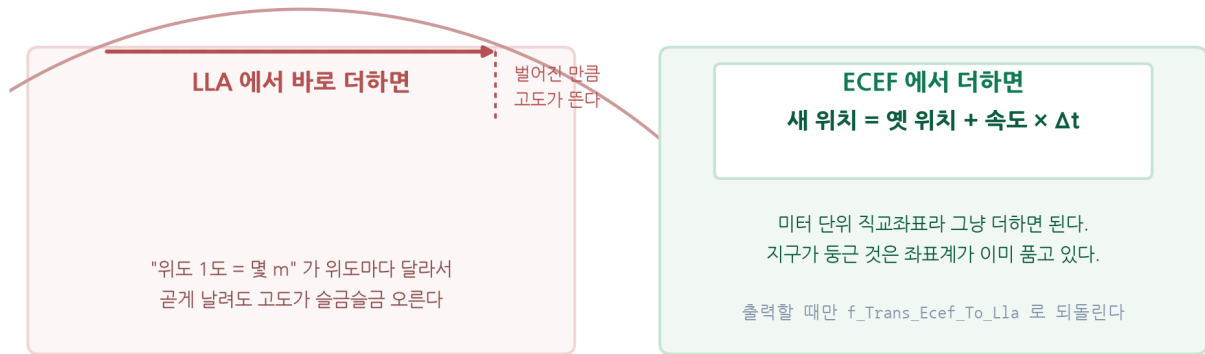
NED 는 지금 내가 서 있는 자리에서 북쪽, 동쪽, 아래쪽을 축으로 잡은 평면 좌표입니다. 기수 방향을 푸는 데 씁니다.

셋의 관계를 한 줄로 정리하면 이렇습니다. 사람은 LLA 로 말하고, 방향은 NED 에서 풀고, 더하기는 ECEF 에서 합니다.

02. 표적 궤적 모의 구현

왜 ECEF 에서 갱신하는가

명세가 정한 조건이기도 하지만, 실제로 계산해 보면 이유가 분명합니다.



18

발표 대본

구현 · 약 55초

명세가 ECEF 에서 갱신하라고 정해 났는데, 왜 그런지 직접 계산해 보면 이유가 분명합니다.

LLA 에서 바로 더하면 어떻게 될까요. "초속 30미터니까 위도를 초당 0.00027도씩 더하자" 이렇게 하는 겁니다.

문제는 위도 1도가 몇 미터인지가 위도마다 다르다는 것입니다. 적도 근처에서 다르고 극지방에서 다릅니다. 그러니까 매 스텝 그 자리의 환산계수를 다시 구해야 하는데, 그걸 제대로 해도 곧은 직선을 따라가려는 성질 때문에 곧게 날린 표적의 고도가 슬금슬금 올라갑니다.

ECEF 에서는 이 문제가 없습니다. 미터 단위 직교좌표라서 새 위치는 옛 위치에 속도 곱하기 델타티를 그냥 더하면 됩니다. 지구가 둥근 것은 좌표계가 이미 품고 있습니다.

출력할 때만 $f_Trans_Ecef_To_Lla$ 로 되돌리면 됩니다.

02. 표적 궤적 모의 구현

동체 속도를 ECEF 속도로 — 과제의 추가 고민

" 표적 동체 속도 값 (V_{heading}) 을 ECEF 좌표계 V_x, V_y, V_z 성분으로 변환하는 방법 " 에 대한 답입니다 .

회전 두 번 — 그래서 속력은 처음 그대로 보존된다



19

발표 대본

구현 · 약 65초

과제에서 추가로 고민해 보라고 한 항목입니다. 동체 속도 V_{heading} 을 ECEF 의 V_x, V_y, V_z 로 어떻게 바꿀 것인가.

결론부터 말씀드리면 회전 두 번입니다.

출발은 동체 좌표입니다. 기수 방향이 x 축이니까 속도는 그냥 $V, 0, 0$ 입니다. 옆도 0, 아래도 0. 이게 제일 쉬운 형태고, 속도를 크기 하나로 받은 덕입니다.

첫 번째 회전이 동체에서 NED 로 가는 것입니다. 자세각 Roll, Pitch, Yaw 를 써서 돌리면 북쪽 성분, 동쪽 성분, 아래 성분이 나옵니다. Yaw 가 180도면 북쪽 성분이 마이너스 V , 동쪽이 0 이 나오겠죠. 남쪽으로 간다는 뜻입니다.

두 번째 회전이 NED 에서 ECEF 로 가는 것입니다. 여기에는 그 지점의 위도와 경도가 들어갑니다. 같은 "북쪽" 이라도 서울에서의 북쪽과 뉴욕에서의 북쪽은 ECEF 에서 전혀 다른 방향이니까요.

두 번 다 회전입니다. 평행이동이 없습니다. 그래서 크기가 안 변합니다.

아래가 그 확인입니다. 60초 동안 601번 속력을 재봤더니 대함은 30.000000000, 대공은 200.000000000 이 그대로 나왔습니다. 소수점 아홉 자리까지입니다.

02. 표적 궤적 모의 구현

코드 ③ — 두 줄이면 끝난다

TargetSimCore/src/TargetSim.c

```
// 동체 속도 (V, 0, 0) 을 자세로 한 번, 위경도로 한 번 돌려 ECEF 속도로 만든다.
// 회전만 거치므로 속력은 그대로 보존된다.
static STRUCT_Coord_Rect f_Tgt_GetVelEcef(const STRUCT_Coord_Attitude *st_Att,
const STRUCT_Coord_Lla *st_Lla, FLOAT64 headingSpeed)
{
    STRUCT_Coord_Rect st_VelNed;

    // 참고 소스는 인자를 Roll, Yaw, Pitch 순서로 받는다.
    st_VelNed = f_Trans_Body_To_Ned(headingSpeed, 0.0, 0.0,
    st_Att->Roll, st_Att->Yaw, st_Att->Pitch);

    // 방향만 옮길 때는 기준점을 원점으로 준다.
    return f_Trans_Ned_To_Ecef(st_VelNed.x, st_VelNed.y, st_VelNed.z,
    0.0, 0.0, 0.0, st_Lla->Lat, st_Lla->Lon);
}
```

왜 기준점이 0 인가

f_Trans_Ned_To_Ecef 는 회전에 더해
기준점만큼 평행이동까지 합니다. 속도는
"위치"가 아니라 "방향"이라 빼야 합니다.

인자 순서를 왜 적어 두었나

제공받은 함수가 Roll, Yaw, Pitch 순서로
받습니다. 자세각 구조체의 필드 순서와
달라, 헷갈리는 자리라 주석을 남겼습니다.

속력 보존 확인

회전행렬은 길이를 바꾸지 않습니다.
60 초 601 개 표본에서 속력을 재보면
200.000000000 m/s 가 601 번 나옵니다.

20

발표 대본

코드 · 약 60초

방금 설명한 게 코드로는 실행문 두 줄입니다.

첫 줄이 동체에서 NED 입니다. headingSpeed 를 첫 인자로 주고 나머지 둘을 0 으로 줍니다. 이게 "기수 방향으로 V" 라는 뜻입니다.

둘째 줄이 NED 에서 ECEF 입니다. 여기서 아래 세 카드에 적은 것들을 조심해야 했습니다.

왼쪽. f_Trans_Ned_To_Ecef 는 원래 표적 위치를 옮기라고 만든 함수라, 회전에 더해서 기준점만큼 평행이동까지 합니다. 그런데 지금 옮기는 건 위치가 아니라 방향입니다. 평행이동이 들어가면 안 됩니다. 그래서 기준점 자리에 0, 0, 0 을 넣었습니다. 이러면 회전만 남습니다.

가운데. 제공받은 f_Trans_Body_To_Ned 는 인자를 Roll, Yaw, Pitch 순서로 받습니다. 그런데 자세각 구조체의 필드는 Roll, Pitch, Yaw 순서입니다. 순서가 다릅니다. 여기서 한 번 틀려서 고생했기 때문에 주석으로 남겨 뒀습니다.

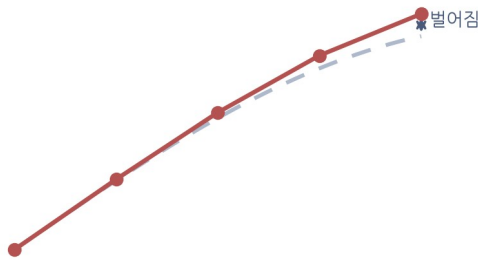
오른쪽이 아까 말씀드린 속력 보존 확인입니다.

02. 표적 궤적 모의 구현

0.1 초를 어떻게 전진시킬 것인가

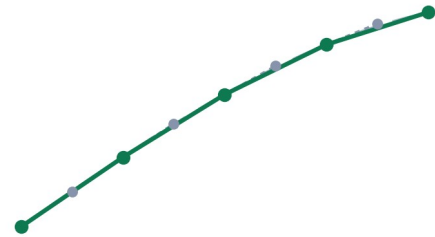
속도가 계속 바뀌는데 한 방향으로만 밀면 안쪽 곡선을 못 따라갑니다.

오일러법 — 출발할 때의 방향으로 쭉



굽은 길을 직선 네 토막으로 근사 — 바깥쪽으로 밀린다

중점법 — 반 걸음 가보고, 거기 방향으로 한 걸음



같은 0.1초인데 오차는 스텝 간격의 제곱으로 줄어든다

21

발표 대본

구현 · 약 55초

위치와 속도를 구했으면 이제 0.1초를 전진시켜야 합니다.

제일 단순한 방법은 왼쪽입니다. 지금 속도로 0.1초 곱해서 더한다. 이것 오일러법이라고 합니다.

직진이면 아무 문제가 없습니다. 그런데 선회 중이면 문제가 생깁니다. 0.1초 동안에도 기수가 계속 돌고 있는데, 출발할 때의 방향으로 쭉 밀어 버리니까 굽은 길을 직선 토막으로 근사하게 됩니다. 결과적으로 늘 바깥쪽으로 밀립니다.

오른쪽이 중점법입니다. 일단 반 걸음만 가봅니다. 거기서 방향을 다시 재고, 그 방향으로 한 걸음을 갑니다. 반 걸음 간 자리가 구간의 평균에 훨씬 가까우니까 오차가 확 줄어듭니다.

계산량은 두 배쯤 되는데, 오차는 두 배가 아니라 간격의 제곱에 비례해서 줄어듭니다. 간격을 절반으로 줄이면 오차는 4분의 1이 됩니다. 이건 뒤에서 숫자로 확인하겠습니다.

02. 표적 궤적 모의 구현

코드 ④ — 한 스텝 전진

TargetSimCore/src/TargetSim.c f_Tgt_Propagate (발체)

```

halfStep = 0.5 * stepTime;

// ① 반 스텝 뒤의 자세
st_AttMid.Roll = st_State->st_Att.Roll + (st_Rate->Roll * halfStep);
st_AttMid.Pitch = st_State->st_Att.Pitch + (st_Rate->Pitch * halfStep);
st_AttMid.Yaw = st_State->st_Att.Yaw + (st_Rate->Yaw * halfStep);

// ② 반 스텝 뒤의 위치 — 스텝 시작 속도로 예측
st_VelStart = f_Tgt_GetVelEcef(&st_State->st_Att, &st_State->st_Lla, headingSpeed);
st_PosMid.x = st_State->st_PosEcef.x + (st_VelStart.x * halfStep);
...

// ③ 그 자리 · 그 자세에서 구한 속도가 이 스텝의 대표 속도
st_LlaMid = f_Trans_Ecef_To_Lla(st_PosMid.x, st_PosMid.y, st_PosMid.z);
st_VelMid = f_Tgt_GetVelEcef(&st_AttMid, &st_LlaMid, headingSpeed);

// ④ 그 속도로 한 스텝을 옮긴다
st_Next.st_PosEcef.x = st_State->st_PosEcef.x + (st_VelMid.x * stepTime);
...

```

왜 반 스텝 위치까지 다시 구하나

자세만 반 스텝 돌리고 위치는 그대로 두면,
지구가 둥근 만큼 방향이 뒤틀립니다.
그러면 수평으로 날아도 고도가 매 스텝
 $d/2R$ 씩 올라갑니다.

짐벌락은 알고 남겨 두었다

오일러각을 직접 적분하므로 Pitch 가
 $\pm 90^\circ$ 근처면 Yaw 축이 속도 방향과 겹쳐
Yaw 기동이 궤적을 바꾸지 못합니다.
코드에 주석으로 명시해 두었습니다.

22

발표 대본

코드 · 약 70초

중점법이 코드로는 이렇게 생겼습니다. 네 덩어리입니다.

- ①번. 자세각을 반 스텝만 돌립니다. 각속도 곱하기 스텝의 절반을 더합니다.
- ②번. 반 스텝 뒤의 위치를 구합니다. 이건 스텝 시작 속도로 그냥 예측합니다.

여기가 처음에 빼먹었던 부분입니다. 원래는 자세만 반 스텝 돌리고 위치는 그대로 두고 속도를 만들었습니다. 그렇게 해도 겉보기에는 잘 돌았는데, 수평으로 날린 표적의 고도가 스텝마다 아주 조금씩 올라갔습니다. 60초에 몇 센티미터쯤.

이유가 지구 곡률이었습니다. 스텝 시작 지점의 위경도로 NED 를 잡으면, 표적은 이미 앞으로 나아갔는데 "북쪽"의 방향은 뒤에 머물러 있는 셈이 됩니다. 그 차이가 매 스텝 d 제곱 나누기 $2R$ 만큼 위로 쌓입니다.

- ③번. 그 자리, 그 자세에서 속도를 다시 구합니다. 이게 이 스텝의 대표 속도입니다.

- ④번. 그 속도로 한 스텝을 옮깁니다.

오른쪽 아래 카드는 알고도 남겨 둔 한계입니다. 오일러각을 직접 적분하니까 Pitch 가 플러스마이너스 90도 근처로 가면 Yaw 축이 속도 방향과 겹쳐서 Yaw 기동이 궤적을 못 바꿉니다. 짐벌락입니다. 사원수를 쓰면 풀리는데 이번 과제 범위를 넘는다고 보고 주석으로만 명시했습니다.

02. 표적 궤적 모의 구현

한 스텝에 벌어지는 일 전부

그리고 계산이 깨졌을 때 상태를 지키는 방법.

`f_Tgt_StepSim` 한 번 = 플랫폼 1 개 + 표적 N 개에 대해 아래를 반복



23

발표 대본

구현 · 약 55초

`f_Tgt_StepSim` 을 한 번 부르면 플랫폼 하나와 표적 N 개에 대해 이 여섯 단계가 됩니다.

지금 시각에 걸린 기동을 찾고, G 를 각속도로 바꾸고, 자세를 반 스텝 돌리고, 그 자세로 속도를 만들고, 그 속도로 위치를 옮기고, ECEF 를 LLA 로 되돌려 같이 보관합니다.

아래 때가 중요한 부분입니다. 이 여섯 단계 어디서든 숫자가 무한대나 NaN 이 되면, 상태를 건드리지 않고 오류 코드만 돌려줍니다.

방법은 간단합니다. 계산을 전부 지역변수 사본에서 하고, 결과가 다 유한한 것을 확인한 다음에야 한 번에 반영합니다. 그래서 스텝 중간에 실패해도 "표적 1은 새 시각, 표적 2는 옛 시각" 같은 상태가 절대 안 나옵니다.

굳이 이렇게 한 이유는, 제공받은 변환 함수들이 값만 돌려줄 뿐 실패를 알려주지 않기 때문입니다. 그래서 결과가 유한한지를 제가 직접 봅니다.

03 시뮬레이션 결과

Simulation Results

- 명세 조건 그대로 60 초를 돌린 결과
- 손으로 계산한 값과 맞는지 숫자로 계산
- 갱신 간격을 바꿔 가며 확인한 정확도

24

발표 대본

장절 · 약 25초

세 번째 장, 결과입니다.

명세 조건 그대로 돌린 궤적을 보여 드리고, 그 숫자가 맞는지 손계산과 맞춰 보겠습니다. 그림이 그럴듯하게 나왔다고 맞는 건 아니니까요.

03. 시뮬레이션 결과

시뮬레이션 조건

「과제 3 내용」 2 항의 값을 그대로 넣었습니다.

	위도 [°]	경도 [°]	고도 [m]	속력 [m/s]	Roll [°]	Pitch [°]	Yaw [°]
플랫폼	32.0	126.0	0	0	0	0	0
대함 표적	32.125	126.03	0	30	0	0	270
대공 표적	32.12	126.0	300	200	0	0	180

시뮬레이션 시간 60 초

갱신 간격 0.1 초 → 601 개 표본
($t=0$ 과 $t=60$ 을 모두 포함)

Yaw 270° 는 서쪽

Yaw 는 북쪽이 0, 시계 방향이 +.
대함은 서쪽, 대공은 남쪽 (180°) 으로 갑니다.

기동 없는 등속 직진

먼저 기동 없이 돌려 순계산과 맞춰 보고,
그 다음 기동을 없었습니다.

플랫폼 속력이 0 이라 $f_{Tgt_Propagate}$ 의 나눗셈 분모가 되는 자리가 없는지 따로 확인했습니다 — 각속도 계산은 기동이 있을 때만 들어갑니다.

25

발표 대본

결과 · 약 45초

과제 문서 2항의 값을 그대로 넣었습니다.

플랫폼은 위도 32도, 경도 126도, 고도 0, 속력 0 입니다. 움직이지 않습니다.

대함 표적은 위도 32.125, 경도 126.03, 고도 0, 초속 30미터, Yaw 270도. 270도면 서쪽입니다.

대공 표적은 위도 32.12, 경도 126.0, 고도 300미터, 초속 200미터, Yaw 180도. 남쪽입니다.

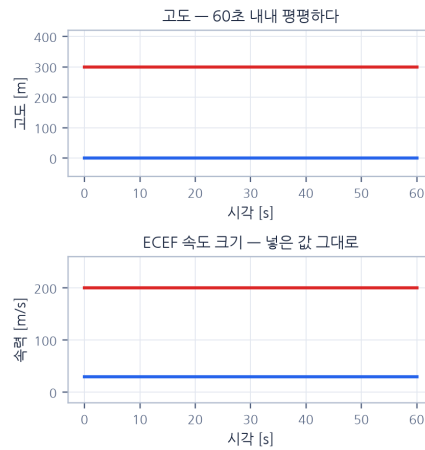
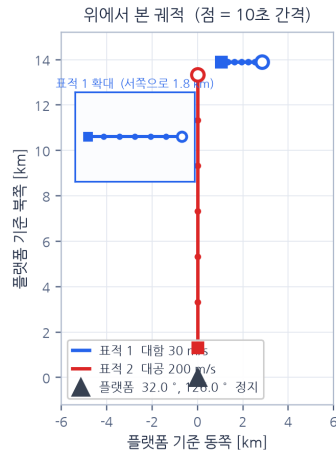
시뮬레이션 시간 60초에 간격 0.1초니까 표본이 601개 나옵니다. 0초와 60초를 둘 다 포함해서 601개입니다.

한 가지 확인했던 것이, 플랫폼 속력이 0 이라는 점입니다. 각속도 계산에 속력이 분모로 들어가니까 0 이면 나눗셈이 터질 수 있는데, 그 계산은 기동이 있을 때만 들어갑니다. 플랫폼은 기동이 없으니 그 자리를 아예 지나가지 않습니다.

03. 시뮬레이션 결과

결과 ① — 60 초 궤적

플랫폼을 원점으로 잡은 동-북 평면입니다. 점 하나가 10 초입니다.



26

발표 대본

결과 · 약 55초

결과입니다. 플랫폼을 원점으로 잡은 동-북 평면이고, 점 하나가 10초입니다.

빨간색 대공 표적이 정남쪽으로 12킬로미터를 내려갔습니다. 초속 200미터 곱하기 60초니까 12킬로미터가 맞습니다.

파란색 대함 표적은 너무 작아서 왼쪽 위에 확대해 넣었습니다. 서쪽으로 1.8킬로미터입니다. 초속 30미터 곱하기 60초입니다.

둘 다 완벽한 직선입니다. 기동을 안 걸었으니 당연히 그래야 합니다.

오른쪽 위가 고도입니다. 60초 내내 평평합니다. 대공은 300미터, 대함은 0미터 그대로입니다. 아까 중점법 이야기할 때 말씀드린 그 고도 상승이 여기서 사라진 것입니다.

오른쪽 아래가 ECEF 속도의 크기입니다. 이것도 평평합니다. 회전만 했으니 크기가 안 변합니다.

03. 시뮬레이션 결과

결과 ② — 손으로 계산해 보면

속력 × 시간, 고도 유지, 위도 유지 — 답을 미리 아는 항목으로 맞춰 봤습니다.

검산 항목	이론값 (손으로 계산)	시뮬레이션 결과	차이
대함 표적이 60초 동안 간 거리	$30 \text{ m/s} \times 60 \text{ s} = 1,800 \text{ m}$	1799.99996 m	-0.00004 m
대공 표적이 60초 동안 간 거리	$200 \text{ m/s} \times 60 \text{ s} = 12,000 \text{ m}$	11999.99995 m	-0.00005 m
대함 고도 (수평 서진)	0 m 유지	0.0000 ~ 0.0000 m	0
대공 고도 (수평 남진)	300 m 유지	300.0000 ~ 300.0000 m	0
대함 위도 (정서쪽으로만)	32.125° 유지	32.125000000° 고정	0
대공 경도 (정남쪽으로만)	126.0° 유지	126.000000000° 고정	0

경로 길이는 0.1초마다 찍힌 601개 점 사이 거리를 전부 더해서 낸 값이다.
12 km 를 날아 오차가 0.05 mm 수준이면, 남은 오차는 계산 방법이 아니라 double 형이 표현할 수 있는 한계에 가깝다.

27

발표 대본

결과 · 약 65초

그림이 그럴듯하다고 맞는 건 아니니까, 답을 미리 아는 항목으로 맞춰 봤습니다.

첫 줄. 대함 표적이 60초 동안 간 거리. 손으로는 초속 30미터 곱하기 60초 해서 1800미터입니다. 시뮬레이션은 1799.99996미터가 나왔습니다. 차이가 0.04밀리미터입니다.

둘째 줄. 대공은 12000미터여야 하는데 11999.99995 가 나왔습니다. 차이 0.05밀리미터.

이 거리는 0.1초마다 찍힌 601개 점 사이의 거리를 전부 더해서 낸 것입니다. 시작점과 끝점 직선거리가 아니라 실제로 지나간 길이입니다.

12킬로미터를 날아서 오차가 0.05밀리미터면, 이건 계산 방법이 틀려서 생긴 오차가 아닙니다. double 형이 표현할 수 있는 한계에 거의 닿은 값입니다.

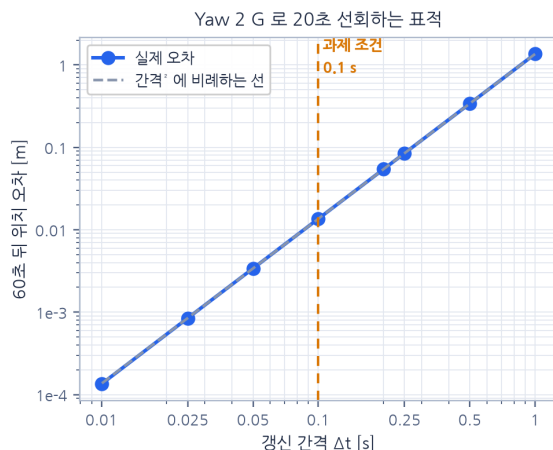
아래 네 줄은 더 단순합니다. 고도는 소수 넷째 자리까지 그대로고, 대함 위도는 서쪽으로만 가니까 32.125도가 아홉 자리까지 고정, 대공 경도는 남쪽으로만 가니까 126도가 고정입니다.

정서쪽으로 가는데 위도가 변하면 안 되고, 정남쪽으로 가는데 경도가 변하면 안 됩니다. 이 두 줄이 좌표변환이 제대로 붙었다는 제일 확실한 증거였습니다.

03. 시뮬레이션 결과

결과 ③ — 0.1 초는 충분한 간격인가

직진은 틀릴 수가 없으니, 가장 많이 휘는 선화 구간으로 확인했습니다.



읽는 법

두 선이 나란하다

간격을 절반으로 줄이면 오차가 1/4 로 준다.
중점법이 제대로 2차 정확도로 들고 있다는 뜻.

0.1초에서 1.4 cm

20초를 2 G 로 꺾는 동안 쌓인 오차가 1.4 cm.
레이다 거리 분해능(수 m)에 비하면 없는 값이다.

직진이면 더 작다

기동이 없을 때는 오차가 10 나노미터 수준 —
사실상 계산 오차만 남는다.

28

발표 대본

결과 · 약 65초

그러면 0.1초라는 간격이 충분한가. 직진은 틀릴 수가 없으니 가장 많이 휘는 경우로 확인했습니다.

대공 표적에 2G 로 20초 선화를 걸어 놓고, 갱신 간격만 1초부터 0.01초까지 바꿔 가며 60초를 돌렸습니다. 기준은 0.001초로 돌린 결과입니다.

왼쪽 그림이 그 결과인데, 양쪽 축 모두 로그입니다. 파란 실선이 실제 오차고 회색 점선이 "간격의 제곱에 비례하는 선" 입니다. 둘이 나란합니다.

나란하다는 건 간격을 절반으로 줄이면 오차가 4분의 1이 된다는 뜻입니다. 중점법이 의도한 대로 2차 정확도로 들고 있다는 증거입니다. 실제로 1초일 때 1.36미터, 0.5초일 때 0.34미터, 0.25초일 때 0.085미터로 정확히 4분의 1씩 줄어듭니다.

과제 조건인 0.1초 자리를 보면 오차가 1.4센티미터입니다. 20초 동안 2G 로 꺾는 내내 쌓인 게 1.4센티미터입니다.

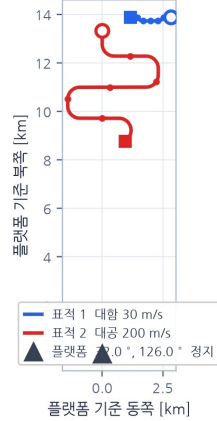
레이다 거리 분해능이 보통 수 미터 단위인 걸 생각하면, 이건 없는 값이라고 봐도 됩니다. 0.1초로 충분합니다.

03. 시뮬레이션 결과

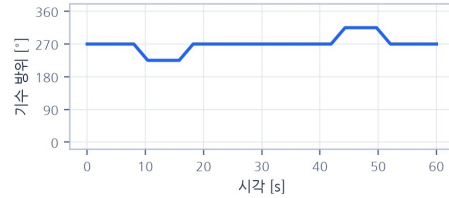
결과 ④ — 기동을 없으면

같은 초기값에 기동 10 줄만 추가했습니다. 초기값 · 시간 · 간격은 명세 그대로입니다.

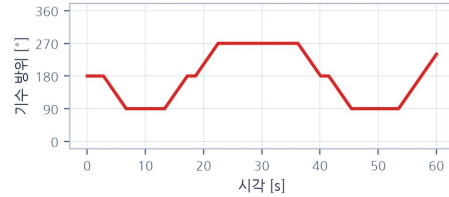
기동 시연 — 표적 2개에 기동 10줄



표적 1 대함 — 계단이 곧 기동 구간



표적 2 대공 — 계단이 곧 기동 구간



29

발표 대본

결과 · 약 50초

마지막으로 기동을 없은 결과입니다.

초기값도, 시뮬레이션 시간도, 갱신 간격도 명세 그대로입니다. 기동표 10줄만 추가했습니다.

빨간색 대공 표적이 남쪽으로 내려가다가 동쪽으로 꺾고, 다시 남쪽, 다시 서쪽, 이렇게 훑고 내려갑니다. 파란 대함은 서쪽으로 가다가 한 번 비켜 갑니다.

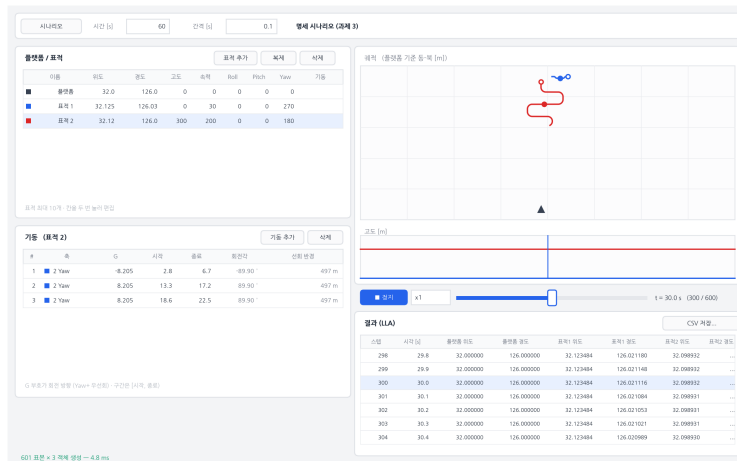
오른쪽 두 그림이 기수 방위각입니다. 계단처럼 생겼는데, 평평한 데가 직진 구간이고 기울어진 데가 기동 구간입니다. 기동표에 적은 시각과 이 계단의 위치가 정확히 맞습니다.

여기서 G 값을 어떻게 정했는지 한 가지만 말씀드리면, 회전 지점이 좌표로 주어졌는데 구조체에는 시각 칸밖에 없어서, Core 를 실제로 돌려 그 좌표에 닿는 시각을 찾아 옮겼습니다. 회전하는 동안에도 표적이 계속 나아가니까 선 회 반경만큼 미리 꺾어야 지정한 자리에서 수평이 됩니다.

03. 시뮬레이션 결과

TargetSim — 값을 바꿔 가며 볼 수 있게

표에서 숫자를 고치면 바로 다시 돌고, 그림에서 시작점과 기수를 끌 수도 있습니다.



고치면 곧바로 다시 돈다

표의 칸을 고치면 0.25 초 뒤 자동으로 다시 계산합니다. 601 표본 $\times$ 3 객체가 5 ms 안에 끝나 기다릴 일이 없습니다.

그림에서 끝아서 배치

시작점과 기수 손잡이를 마우스로 끌면
위도 · 경도 · Yaw 가 표에 그대로
반영됩니다.

결과는 표 · CSV · 그림

시각마다의 LLA 를 표로 보고, 슬라이더로
되감아 보고, CSV 로 내보냅니다.
시나리오는 .tsim 파일로 저장합니다.

발표 대본

값을 코드에 박아 놓고 다시 컴파일하는 건 불편해서 화면을 붙였습니다.

왼쪽 위 표에서 표적의 초기값을 고치면 0.25초 뒤에 자동으로 다시 계산합니다. 601 표본에 객체 3개면 5밀리초 안에 끝나서 기다릴 일이 없습니다. 왼쪽 아래가 기동표고, 회전각과 선회 반경은 입력한 G 와 시간으로 계산해서 같이 보여 줍니다.

오른쪽 위가 꺾적입니다. 여기서 시작점 표식을 마우스로 끌면 위도와 경도가 바뀌고, 기수 손잡이를 끌면 Yaw 가 바뀝니다. 표에 숫자로 넣는 것보다 시나리오를 잡을 때 훨씬 빠릅니다.

아래가 결과 표인데 시각마다의 LLA 가 그대로 나옵니다. 슬라이더로 되감아 볼 수 있고, CSV 로 내보낼 수 있습니다. 시나리오 자체는 텍스트 파일로 저장해서 다시 불러올 수 있게 했습니다.

결과 · 약 50초

03. 시뮬레이션 결과

정리

요구사항과 결과를 한 장에 놓고 보겠습니다.

과제 요구사항	이렇게 했습니다	확인
표적 10개 · 기동 30개	고정 배열로 상한을 두고 범위 밖은 오류 코드로 거부	완료
위치는 LLA, 속도는 V_heading	ST_TargetInit 에 그대로. 방향은 자세각이 담당	완료
기동 = G · 축 · 시작 · 종료	ST_TargetManeuver 네 필드. 구간 겹침도 검사	완료
ECEF 에서 상태 갱신	위치 · 속도 모두 ECEF, 출력용 LLA 를 함께 보관	완료
V_heading → Vx, Vy, Vz (추가 고민)	동체 → NED → ECEF 두 번 회전. 속도 보존 확인	완료
제공 좌표변환 코드 사용	CoordinateTransform.c / matrixCalcLib.c 원본 그대로	완료
결과를 LLA 로 출력하고 그림으로	표 · CSV · 궤적 그림 · 고도 그림	완료

남겨 둔 것

- Pitch 가 $\pm 90^\circ$ 근처면 짐벌락 — 오일러각 대신 사원수를 쓰면 풀립니다
- 중력 · 항력 없이 기수 방향으로만 나아가는 운동학 모델입니다

다음에 이어 붙일 것

- 이 궤적에 잡음을 얹으면 곧바로 추적 필터 입력이 됩니다
- Part 2 의 좌표변환을 붙이면 안테나 기준 거리 · 방위 · 고각이 나옵니다

31

발표 대본

정리 · 약 65초

요구사항과 결과를 한 장에 놓고 보겠습니다.

표적 10개와 기동 30개는 고정 배열로 상한을 두고, 범위를 넘으면 오류 코드로 거부했습니다.

위치는 LLA, 속도는 V_heading 하나로 받았고 방향은 자세각이 담당합니다.

기동은 축, G, 시작, 종료 네 필드로 정의하고 구간 겹침까지 검사합니다.

상태 갱신은 ECEF 에서 하고, 출력용 LLA 를 같이 보관합니다.

추가 고민 항목이었던 V_heading 에서 Vx, Vy, Vz 변환은 동체에서 NED, NED 에서 ECEF 로 두 번 회전했고 속력이 보존되는 것을 601개 표본으로 확인했습니다.

좌표변환 코드는 제공받은 것을 원본 그대로 썼습니다.

결과는 표와 CSV, 궤적 그림, 고도 그림으로 냈습니다.

남겨 둔 것도 말씀드리겠습니다. Pitch 가 90도 근처로 가면 짐벌락이 있고, 이건 사원수를 써야 풀립니다. 그리고 이 모델은 중력이나 항력이 없이 기수 방향으로만 나아가는 운동학 모델입니다. 실제 탄도를 그리려면 힘을 넣어야 합니다.

반대로, 이 궤적에 잡음만 얹으면 바로 추적 필터 입력이 되고, Part 2 에서 만든 좌표변환을 붙이면 안테나 기준 거리와 방위각, 고각이 나옵니다. 다음 과제로 이어지는 부분입니다.

감사합니다

질문 주시면 답변드리겠습니다

32

발표 대본

마무리 · 약 15초

이상입니다. 감사합니다.

질문 있으시면 답변드리겠습니다.